

Módulo 1.1: Introducción a Python desde Cero

Curso: **Machine Learning con Python** (IFCD093PO)

Duración estimada: 10 horas

Objetivos del Módulo

En este módulo, aprenderás a programar en Python desde cero. No se requiere experiencia previa. Al finalizar, serás capaz de:

- ✓ Escribir y ejecutar tus primeros programas en Python.
- ✓ Entender y usar variables y los tipos de datos fundamentales.
- ✓ Trabajar con las estructuras de datos más importantes: listas, diccionarios, tuplas y sets.
- ✓ Controlar el flujo de tus programas con condicionales y bucles.
- ✓ Crear tus propias funciones para reutilizar código.
- ✓ Comprender los conceptos básicos de la Programación Orientada a Objetos (POO).

Este es el cimiento sobre el que construiremos todo lo demás. ¡Vamos a empezar!

Tabla de Contenidos

- ¿Qué es Python y por qué usarlo?
- Tu Primer Programa: "Hola, Mundo"
- Variables y Tipos de Datos
- Estructuras de Datos
 - Listas
 - Diccionarios
 - Tuplas
 - Sets
- Control de Flujo
 - Condicionales (if, elif, else)
 - Bucles (for, while)
- Funciones
- Programación Orientada a Objetos (POO)
- Resumen y Próximos Pasos

1. ¿Qué es Python y por qué usarlo?

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general. ¿Qué significa todo eso?

- Alto nivel:** Su sintaxis es muy parecida al lenguaje humano (inglés), lo que lo hace fácil de leer y escribir.
- Interpretado:** No necesitas compilar tu código. Un intérprete lee y ejecuta el código línea por línea, lo que agiliza el desarrollo.
- Propósito general:** Sirve para casi todo: desarrollo web, ciencia de datos, inteligencia artificial, scripting, automatización, etc.

¿Por qué es el lenguaje #1 para Machine Learning?

- Sintaxis Simple:** Permite concentrarse en los problemas de ML en lugar de en la complejidad del lenguaje.
- Ecosistema de Librerías:** Tiene librerías increíbles como **NumPy**, **Pandas**, **Scikit-learn**, **TensorFlow** y **PyTorch** que hacen el trabajo pesado por nosotros.
- Comunidad Gigante:** Millones de desarrolladores usan Python. Si tienes un problema, es casi seguro que alguien ya lo ha resuelto.
- Flexibilidad:** Se integra fácilmente con otros lenguajes y tecnologías.

2. Tu Primer Programa: "Hola, Mundo"

La tradición al aprender un nuevo lenguaje es empezar con un "Hola, Mundo". En Python, es increíblemente simple.

Usamos la función `print()` para mostrar texto en la pantalla.

```
In [ ]: # Este es un comentario. Python lo ignora.
# La función print() muestra un mensaje en la pantalla.

print("¡Hola, Mundo!")
print("Bienvenido al curso de Machine Learning con Python.")
print(23)
```

3. Variables y Tipos de Datos

Una **variable** es como una caja con una etiqueta donde guardamos información. Podemos ponerle un nombre y asignarle un valor con el signo `=`.

Python tiene varios **tipos de datos** incorporados. Los más importantes son:

- **int** (Enteros): Números sin decimales (ej: 10, -5, 0).
- **float** (Flotantes): Números con decimales (ej: 3.14, -0.5).
- **str** (Strings o Cadenas): Texto, siempre entre comillas `"` o `'`.
- **bool** (Booleanos): Representan verdad o falsedad. Solo pueden ser `True` o `False`.

```
In [ ]: # Asignación de variables
nombre = "Ana"          # str (string)
edad = 30               # int (entero)
altura = 1.65           # float (flotante)
es_estudiante = True    # bool (booleano)

# Podemos imprimir las variables para ver su valor
print("Nombre:", nombre)
print("Edad:", edad)
print("Altura:", altura)
print("¿Es estudiante?", es_estudiante)

# La función type() nos dice de qué tipo es una variable
print("\nTipos de datos:")
print("El tipo de 'nombre' es:", type(nombre))
print("El tipo de 'edad' es:", type(edad))
```

Operaciones con Variables

Podemos realizar operaciones matemáticas con las variables numéricas y concatenar (unir) las cadenas de texto.

```
In [ ]: # Operaciones numéricas
año_actual = 2025
año_nacimiento = 1995
edad_calculada = año_actual - año_nacimiento
print(f"Si naciste en {año_nacimiento}, en {año_actual} tienes {edad_calculada} años.")

# Concatenación de strings
nombre_completo = "Juan" + " " + "Pérez"
print("Nombre completo:", nombre_completo)

# f-strings: la forma moderna y recomendada de formatear texto
print(f"Hola, me llamo {nombre_completo} y tengo {edad_calculada} años.")
```

✓ Ejercicio 1: Calculadora de IMC

El Índice de Masa Corporal (IMC) se calcula con la fórmula: `peso / (altura2)`.

1. Crea dos variables: `peso` (en kg) y `altura` (en metros).
2. Calcula el IMC y guárdalo en una variable `imc`.
3. Imprime el resultado con un mensaje claro usando un f-string.

```
In [ ]: # Tu código aquí
peso = 75 # en kg
altura = 1.80 # en metros

# Calcula el IMC
imc = peso / altura ** 2
# Imprime el resultado
print(f"El IMC es: {imc:.2f}")
```

► 💡 **Solución (haz clic para ver)**

🧱 4. Estructuras de Datos

A menudo necesitamos guardar colecciones de datos, no solo valores individuales. Python nos ofrece varias estructuras para ello.

4.1 Listas

Una **lista** es una colección **ordenada** y **modificable** de elementos. Se definen con corchetes `[]`.

- **Ordenada:** Los elementos mantienen el orden en que los añadiste.
- **Modificable:** Puedes añadir, eliminar o cambiar elementos después de crearla.

```
In [ ]: # Creación de una lista
frutas = ["manzana", "banana", "cereza", "naranja"]
print("Lista de frutas:", frutas)

# Acceder a elementos (la indexación empieza en 0)
primera_fruta = frutas[0] # El primer elemento
ultima_fruta = frutas[-1] # El último elemento
print(f"La primera fruta es '{primera_fruta}' y la última es '{ultima_fruta}'.")

# Slicing (sublistas): lista[inicio:fin]
primeras_dos = frutas[0:2]
print("Las dos primeras frutas son:", primeras_dos)

# Modificar un elemento
```

```

frutas[1] = "plátano" # Cambiamos 'banana' por 'plátano'
print("Lista modificada:", frutas)

# Añadir elementos
frutas.append("kiwi") # Añade al final
print("Lista con kiwi:", frutas)

# Eliminar elementos
frutas.remove("cereza")
print("Lista sin cereza:", frutas)

# Longitud de la lista
print(f"Ahora hay {len(frutas)} frutas en la lista.")

```

✓ Ejercicio 2: Lista de la compra

1. Crea una lista llamada `lista_compra` con tres productos que necesites.
2. Añade un cuarto producto a la lista.
3. Cambia el segundo producto por otro que prefieras.
4. Imprime la lista final y su longitud.

In []: # Tu código aquí

► 💡 Solución

4.2 Diccionarios

Un **diccionario** es una colección **no ordenada** de pares `clave: valor`. Se definen con llaves `{}`.

- **No ordenada** (en versiones antiguas de Python): No puedes confiar en el orden.
- **Modificable**: Puedes añadir, cambiar o eliminar pares.
- **Claves únicas**: Cada clave debe ser única.

```

In [ ]: # Creación de un diccionario
persona = {
    "nombre": "Carlos",
    "edad": 42,
    "ciudad": "Madrid",
    "profesion": "Científico de Datos"
}
print("Diccionario de persona:", persona)

# Acceder a valores a través de su clave
print(f"Carlos vive en {persona['ciudad']}.")

# Modificar un valor
persona["edad"] = 43
print("Edad actualizada:", persona["edad"])

# Añadir un nuevo par clave-valor
persona["email"] = "carlos@email.com"
print("Diccionario con email:", persona)

# Obtener todas las claves y valores
print("Claves:", persona.keys())
print("Valores:", persona.values())

```

✓ Ejercicio 3: Información de un libro

1. Crea un diccionario que represente un libro, con las claves `título`, `autor` y `año`.
2. Añade una nueva clave `genero`.
3. Imprime una frase que resuma la información del libro.

In []: # Tu código aquí

► 💡 Solución

4.3 Tuplas

Una **tupla** es una colección **ordenada** e **inmutable**. Se definen con paréntesis `()`.

- **Inmutable**: Una vez creada, **no puedes cambiarla**. Ni añadir, ni eliminar, ni modificar elementos.

Se usan para datos que no deben cambiar, como coordenadas, colores RGB, o registros de una base de datos.

```

In [ ]: # Creación de una tupla
coordenadas = (40.4168, -3.7038) # Coordenadas de Madrid
print("Coordenadas:", coordenadas)

# Acceder a elementos (igual que las listas)
latitud = coordenadas[0]
longitud = coordenadas[1]
print(f"Latitud: {latitud}, Longitud: {longitud}")

# Desempaquetado de tuplas (muy útil)
lat, lon = coordenadas

```

```
print(f"Latitud (desempaquetada): {lat}, Longitud: {lon}")

# Intentar modificar una tupla dará un error
try:
    coordenadas[0] = 41.0
except TypeError as e:
    print(f"\nError al intentar modificar la tupla: {e}")
```

```
In [ ]: fecha = (2,29,2024)
mm, dd, aaaa = fecha
print(f"La fecha es: {dd}-{mm}-{aaaa}")
```

4.4 Sets (Conjuntos)

Un **set** es una colección **no ordenada** de elementos **únicos**. Se definen con llaves `{}` como los diccionarios, pero sin pares clave-valor.

- **Únicos:** No puede haber elementos duplicados. Si intentas añadir un elemento que ya existe, simplemente se ignora.

Son muy útiles para eliminar duplicados de una lista o para operaciones matemáticas de conjuntos (unión, intersección).

```
In [ ]:

In [ ]: # Creación de un set a partir de una lista con duplicados
numeros_con_duplicados = [1, 2, 2, 3, 4, 4, 4, 5]
numeros_unicos = set(numeros_con_duplicados)

print("Lista original:", numeros_con_duplicados)
print("Set (elementos únicos):", numeros_unicos)

# Operaciones de conjuntos
set_A = {1, 2, 3, 4}
set_B = {3, 4, 5, 6}

union = set_A.union(set_B) # Todos Los elementos
interseccion = set_A.intersection(set_B) # Elementos en común

print("\nUnión de A y B:", union)
print("Intersección de A y B:", interseccion)
```

5. Control de Flujo

El control de flujo nos permite tomar decisiones y repetir acciones en nuestro código.

5.1 Condicionales (if, elif, else)

Permiten ejecutar un bloque de código solo si se cumple una condición.

```
In [ ]: edad = 65

if edad < 18:
    print("Es menor de edad.")
elif edad >= 18 and edad < 65:
    print("Es un adulto.")
else:
    print("Es un jubilado.")

# Ejemplo con booleanos
esta_lloviendo = False

if esta_lloviendo:
    print("No te olvides el paraguas.")
else:
    print("¡Disfruta del buen tiempo!")
```

```
In [ ]: a = 20
b = 20
if a > b:
    print("a es mayor que b")
elif b > a:
    print("b es mayor que a")
else:
    print("a y b son iguales")
```

5.2 Bucles (for, while)

Los bucles nos permiten repetir un bloque de código varias veces.

Bucle `for`

Se usa para **iterar sobre una secuencia** (como una lista, tupla o string).

```
In [ ]: # Iterar sobre una lista
frutas = ["manzana", "banana", "cereza", "pera"]
print("Iterando sobre una lista:")
```

```

print(frutas[0])
print(frutas[1])
print(frutas[2])

for fruta in frutas:
    print(f"- {fruta}")

# Iterar un número específico de veces con range()
print("\nContando hasta 4:")
for i in range(10): # range(5) genera números de 0 a 4
    print(i)

for a in range(1,11):
    print(f"Tabla del {a}")
    for b in range(1,11):
        r = a * b
        print(f"{a} X {b} = {r}")

```

Bucle while

Se usa para repetir un bloque de código **mientras se cumpla una condición**.

```

In [ ]: contador = 0

print("Bucle while:")
while contador < 5:
    print(contador)
    contador = contador + 1 # ¡Importante! Hay que modificar la condición para no crear un bucle infinito

print("Fin del bucle while.")

n = 1
while n < 11:
    print(f"Tabla de multiplicar del {n}")
    m = 1
    while m < 11:
        print(f"{n} X {m} = {n*m}")
        m = m + 1
    n = n + 1

```

✓ Ejercicio 4: Suma de números pares

Usa un bucle `for` y un condicional `if` para sumar todos los números pares del 1 al 20.

```

In [ ]: # Tu código aquí
suma_pares = 0

# El 21 es para que incluya el 20
for numero in range(1, 21):
    # Comprueba si el número es par
    # El operador % (módulo) da el resto de una división
    if numero % 2 == 0:
        suma_pares = suma_pares + numero

print(f"La suma de los números pares del 1 al 20 es: {suma_pares}")

```

► 💡 Solución



6. Funciones

Una **función** es un bloque de código reutilizable que realiza una tarea específica. Se definen con la palabra clave `def`.

Las funciones nos ayudan a:

- **Organizar** el código.
- **Reutilizar** lógica sin copiar y pegar.
- Hacer el código más **legible**.

```

In [ ]: # Definición de una función simple
def saludar(nombre):
    """Esta función recibe un nombre y devuelve un saludo."""
    return f"¡Hola, {nombre}! ¿Cómo estás?"

# Llamada a la función
mensaje_para_ana = saludar("Ana")
print(mensaje_para_ana)
print(saludar("Juan"))
mensaje_para_pedro = saludar("Pedro")
print(mensaje_para_pedro)

# Función con múltiples parámetros y valor por defecto
def calcular_precio_final(precio_base, iva=0.21):
    """Calcula el precio final añadiendo el IVA."""
    return precio_base * (1 + iva)

precio_producto_A = 100

```

```

precio_final_A = calcular_precio_final(precio_producto_A)
print(f"\nEl precio final de un producto de {precio_producto_A}€ es {precio_final_A:.2f}€ (con 21% de IVA).")

precio_producto_B = 50
precio_final_B = calcular_precio_final(precio_producto_B, iva=0.10) # IVA reducido
print(f"El precio final de un producto de {precio_producto_B}€ es {precio_final_B:.2f}€ (con 10% de IVA).")

def hola():
    return "hola caracola"

print(hola())
print(hola())

```

✓ Ejercicio 5: Función de Área de un Círculo

El área de un círculo se calcula como $\pi * \text{radio}^2$.

1. Importa la constante `pi` del módulo `math`.
2. Crea una función `area_circulo` que reciba el `radio`.
3. La función debe devolver el área calculada.
4. Prueba la función con diferentes radios.

```

In [ ]: # Tu código aquí
# 1. Importar pi
# import math
pi = 3.1416
# 2. Definir la función
def area_circulo(radio):
    return pi * radio ** 2
# 4. Probar la función
print("""Calcula el área
de un círculo dado
su radio.""")
print(area_circulo(10))
print(area_circulo(0))
print(math.pi)

```

► 💡 Solución

🏠 7. Programación Orientada a Objetos (POO)

La POO es un paradigma para estructurar nuestro código. En lugar de solo funciones y variables, creamos **objetos** que combinan **datos (atributos)** y **comportamientos (métodos)**.

Una **clase** es como un plano o una plantilla para crear objetos. Un **objeto** (o instancia) es una entidad creada a partir de una clase.

```

In [ ]: # Definición de la clase 'Coche'
class Coche:
    # El método __init__ es el constructor. Se llama al crear un objeto.
    def __init__(self, marca, modelo, año):
        # Atributos (datos del objeto)
        self.marca = marca
        self.modelo = modelo
        self.año = año
        self.velocidad = 0 # Velocidad inicial

    # Métodos (comportamientos del objeto)
    def acelerar(self, cantidad):
        self.velocidad += cantidad # self.velocidad = self.velocidad + cantidad
        print(f"El {self.modelo} acelera. Velocidad actual: {self.velocidad} km/h")

    def frenar(self, cantidad):
        self.velocidad -= cantidad
        if self.velocidad < 0:
            self.velocidad = 0
        print(f"El {self.modelo} frena. Velocidad actual: {self.velocidad} km/h")

    def describir(self):
        return f"Coche: {self.marca} {self.modelo} del año {self.año}"

# Creación de objetos (instancias) de la clase Coche
mi_coche = Coche("Toyota", "Corolla", 2022)
coche_de_ana = Coche("Ford", "Mustang", 1968)
coche_de_pepe = Coche("Fiat", "500", 1958)

# Usar los objetos
print(mi_coche.describir())
mi_coche.acelerar(50)
mi_coche.acelerar(20)
mi_coche.frenar(20)

print("\n" + coche_de_ana.describir())
coche_de_ana.acelerar(100)

```

✓ Ejercicio 6: Clase Rectángulo

1. Crea una clase `Rectangulo`.

2. El constructor `__init__` debe recibir `base` y `altura`.
3. Crea un método `calcular_area()` que devuelva el área (`base * altura`).
4. Crea un método `calcular_perimetro()` que devuelva el perímetro (`2 * base + 2 * altura`).
5. Crea un objeto de esta clase y prueba sus métodos.

In []: `# Tu código aquí`

►  Solución

8. Resumen y Próximos Pasos

 ¡Felicidades! Has completado tu introducción a Python

✓ Lo que has aprendido:

1. Fundamentos

- Escribir código Python y usar `print()`.
- Crear y usar variables (`int`, `float`, `str`, `bool`).

2. Estructuras de Datos

- **Listas:** colecciones ordenadas y modificables.
- **Diccionarios:** pares `clave: valor`.
- **Tuplas:** colecciones ordenadas e inmutables.
- **Sets:** colecciones de elementos únicos.

3. Control de Flujo

- Tomar decisiones con `if`, `elif`, `else`.
- Repetir código con bucles `for` y `while`.

4. Organización del Código

- Crear funciones reutilizables con `def`.
- Entender la estructura básica de una clase en POO.

Próximo Módulo: Librerías Fundamentales para Machine Learning

Ahora que tienes las bases de Python, estás listo para aprender las herramientas que hacen de Python el lenguaje rey para la ciencia de datos:

- **NumPy:** Para cálculos numéricos y matemáticos a gran velocidad.
- **Pandas:** Para manipular y analizar datos tabulares (como hojas de cálculo).